

GUIA DE ESTRATÉGIA

Docker · Ambientes Dev & Prod

NextJS · Python · Rust · PostgreSQL · Supabase

1. Visão Geral da Estratégia

A diferença fundamental entre os dois ambientes dita toda a arquitetura:

- Dev (Windows): todas as linguagens no host. Docker somente para o PostgreSQL. Porta 5432 exposta para o host.
- Prod (Linux): tudo containerizado. PostgreSQL sem porta exposta — isolado na rede interna Docker.
- Prod Supabase: banco gerenciado na nuvem. Sem container de banco em prod — connection string via variável de ambiente.

O mecanismo de merge do Docker Compose permite manter docker-compose.yml como base imutável e aplicar overrides por ambiente com a flag -f, sem nunca editar o arquivo base.

2. Estrutura de Arquivos

```
projeto/
├── docker-compose.yml           # base compartilhada (nunca editar diretamente)
├── docker-compose.dev.yml       # override: dev Windows
├── docker-compose.prod.local.yml # override: prod servidor local
├── docker-compose.prod.supabase.yml # override: prod Supabase
├── .env.dev                     # secrets dev [gitignore]
├── .env.prod.local              # secrets prod local [gitignore]
├── .env.prod.supabase           # secrets Supabase [gitignore]
├── .gitignore
├── package.json                 # scripts de controle
├── scripts/
│   ├── backup.sh
│   ├── restore.sh
│   └── healthcheck.sh
└── apps/
```

```
|— nextjs/  
|— python/  
└─ rust/
```

3. Variáveis de Ambiente

3.1 .env.dev — Desenvolvimento

```
# .env.dev  
DB_USER=r2-user  
DB_PASS=r2-password  
DB_NAME=r2-database  
DB_SCHEMA=r2-schema  
DB_PORT=5432  
DATABASE_URL=postgresql://r2-user:r2-password@localhost:5432/r2-database  
DATABASE_URL_SCHEMA=postgresql://r2-user:r2-password@localhost:5432/r2-  
database?schema=r2-schema
```

3.2 .env.prod.local — Produção Servidor Local

```
# .env.prod.local  
# — Banco local (container na rede interna) —————  
DB_USER=r2-user  
DB_PASS=r2-password  
DB_NAME=r2-database  
DB_SCHEMA=r2-schema  
DB_PORT=5432  
DATABASE_URL=postgresql://r2-user:r2-password@postgres:5432/r2-database  
DATABASE_URL_SCHEMA=postgresql://r2-user:r2-password@postgres:5432/r2-  
database?schema=r2-schema  
  
# — Banco remoto via rede local (outro servidor) —————  
REMOTE_DB_HOST=192.168.1.100  
REMOTE_DB_USER=r2-user  
REMOTE_DB_PASS=r2-password  
REMOTE_DB_NAME=r2-database  
REMOTE_DB_SCHEMA=r2-schema  
REMOTE_DATABASE_URL=postgresql://r2-user:r2-password@192.168.1.100:5432/r2-database  
REMOTE_DATABASE_URL_SCHEMA=postgresql://r2-user:r2-password@192.168.1.100:5432/r2-  
database?schema=r2-schema
```

3.3 .env.prod.supabase — Produção Supabase

```

# .env.prod.supabase
# — Banco local (fallback / migration)
DB_USER=r2-user
DB_PASS=r2-password
DB_NAME=r2-database
DB_SCHEMA=r2-schema
DB_PORT=5432
DATABASE_URL=postgresql://r2-user:r2-password@postgres:5432/r2-database
DATABASE_URL_SCHEMA=postgresql://r2-user:r2-password@postgres:5432/r2-
database?schema=r2-schema

# — Supabase (banco principal)
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_ANON_KEY=<anon-public-key>
SUPABASE_SERVICE_KEY=<service-role-secret-key>
SUPABASE_DB_HOST=db.<project-ref>.supabase.co
SUPABASE_DB_PORT=5432
SUPABASE_DB_USER=postgres
SUPABASE_DB_PASS=<supabase-db-password>
SUPABASE_DB_NAME=postgres
SUPABASE_DATABASE_URL=postgresql://postgres:<password>@db.<ref>.supabase.co:5432/postgres
SUPABASE_DATABASE_URL_SCHEMA=postgresql://postgres:<password>@db.<ref>.supabase.co:5432/postgres?schema=r2-schema

# — Qual banco usar como primário
PRIMARY_DB=supabase # valores: local | supabase

```

⚠ **SUPABASE_SERVICE_KEY** nunca deve ir para o cliente (browser). Use somente em contexto server-side (Next.js API routes, Python, Rust).

4. Arquivos Docker Compose

4.1 docker-compose.yml — Base

O arquivo base não define o serviço postgres. Como o Supabase não usa container de banco, postgres foi movido para os overrides que precisam dele (dev e prod-local). Isso evita qualquer herança indesejada no override do Supabase.

```

# docker-compose.yml — volumes e rede compartilhada
# postgres NAO está aqui — definido em cada override que precisa dele

volumes:
  pgdata:

```

```
  name: r2-pgdata
  certdata:
    name: r2-certdata

networks:
  r2-net:
    name: r2-net
```

4.2 docker-compose.dev.yml — Dev Windows

Define o postgres completo e expõe a porta 5432 para o host. As linguagens rodam no Windows e conectam via localhost:5432.

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: r2-postgres
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
      POSTGRES_DB: ${DB_NAME}
    ports:
      - "5432:5432"          # expoe para o host
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    restart: unless-stopped
    env_file: .env.dev
    networks: [r2-net]

# NextJS, Python e Rust rodam no host — sem containers para linguagens
# Conectam ao banco via localhost:5432
```

✓ O arquivo init.sql cria o schema r2-schema e concede permissões ao r2-user na primeira inicialização do volume.

scripts/init.sql

```
-- Executado automaticamente na primeira criacao do volume
CREATE SCHEMA IF NOT EXISTS "r2-schema";
GRANT ALL PRIVILEGES ON SCHEMA "r2-schema" TO "r2-user";
ALTER USER "r2-user" SET search_path TO "r2-schema", public;
```

4.3 docker-compose.prod.local.yml — Prod Servidor Local

Define o postgres completo sem porta exposta e todas as apps. Banco acessível apenas pela rede interna Docker.

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: r2-postgres
    environment:
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASS}
      POSTGRES_DB: ${DB_NAME}
    volumes:
      - pgdata:/var/lib/postgresql/data
      - ./scripts/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER} -d ${DB_NAME}"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 10s
    restart: unless-stopped
    env_file: .env.prod.local
    networks: [r2-net]
    # SEM ports - rede interna apenas

  nextjs:
    build: ./apps/nextjs
    container_name: r2-nextjs
    restart: unless-stopped
    env_file: .env.prod.local
    depends_on:
      postgres:
        condition: service_healthy

  python:
    build: ./apps/python
    container_name: r2-python
    restart: unless-stopped
```

```
env_file: .env.prod.local
depends_on:
  postgres:
    condition: service_healthy

rust:
  build: ./apps/rust
  container_name: r2-rust
  restart: unless-stopped
  env_file: .env.prod.local
  depends_on:
    postgres:
      condition: service_healthy

nginx:
  image: nginx:alpine
  container_name: r2-nginx
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - certdata:/etc/letsencrypt
  depends_on: [nextjs]

volumes:
  certdata:
    name: r2-certdata
```

4.4 docker-compose.prod.supabase.yml — Prod Supabase

Sem nenhum serviço de banco. O arquivo base não define postgres, então este override sobe apenas as apps e o nginx. As conexões com o Supabase são feitas exclusivamente via variáveis de ambiente `SUPABASE_DATABASE_URL`.

✓ Este compose não cria, não altera e não conecta a nenhum container de banco. O Supabase é externo e já está rodando.

```
services:
  # Sem postgres — banco e o Supabase (externo, já existente)

  nextjs:
    build: ./apps/nextjs
```

```
    container_name: r2-nextjs
    restart: unless-stopped
    env_file: .env.prod.supabase
    networks: [r2-net]
    # Sem depends_on — banco e externo

python:
    build: ./apps/python
    container_name: r2-python
    restart: unless-stopped
    env_file: .env.prod.supabase
    networks: [r2-net]

rust:
    build: ./apps/rust
    container_name: r2-rust
    restart: unless-stopped
    env_file: .env.prod.supabase
    networks: [r2-net]

nginx:
    image: nginx:alpine
    container_name: r2-nginx
    restart: unless-stopped
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./nginx.conf:/etc/nginx/nginx.conf:ro
      - certdata:/etc/letsencrypt
    networks: [r2-net]
    depends_on: [nextjs]
```

5. Scripts package.json

Todos os comandos operacionais centralizados no package.json da raiz do projeto. Instale cross-env e dotenv-cli como devDependencies para compatibilidade Windows/Linux.

```
npm install -D cross-env dotenv-cli
```

5.1 package.json completo

```
{
```

```

"name": "r2-project",
"scripts": {

  "_comment_dev": "—— DESENVOLVIMENTO ——",
  "dev:up": "docker compose -f docker-compose.yml -f docker-compose.dev.yml -
-env-file .env.dev up -d",
  "dev:down": "docker compose -f docker-compose.yml -f docker-compose.dev.yml
down",
  "dev:stop": "docker compose -f docker-compose.yml -f docker-compose.dev.yml
stop",
  "dev:restart": "docker compose -f docker-compose.yml -f docker-compose.dev.yml
restart postgres",
  "dev:logs": "docker compose -f docker-compose.yml -f docker-compose.dev.yml
logs -f postgres",
  "dev:ps": "docker compose -f docker-compose.yml -f docker-compose.dev.yml
ps",
  "dev:reset": "npm run dev:down -- -v && npm run dev:up",

  "_comment_prod_local": "—— PRODUÇÃO LOCAL
——",
  "prod:up": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml --env-file .env.prod.local up -d",
  "prod:down": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml down",
  "prod:stop": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml stop",
  "prod:restart": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml restart",
  "prod:logs": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml logs -f",
  "prod:ps": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml ps",
  "prod:pull": "docker compose -f docker-compose.yml -f docker-
compose.prod.local.yml pull",
  "prod:deploy": "npm run prod:pull && npm run prod:down && npm run prod:up",

  "_comment_supabase": "—— PRODUÇÃO SUPABASE ——",
  "supa:up": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml --env-file .env.prod.supabase up -d",
  "supa:down": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml down",
  "supa:stop": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml stop",
  "supa:restart": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml restart",
  "supa:logs": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml logs -f",
  "supa:deploy": "docker compose -f docker-compose.yml -f docker-
compose.prod.supabase.yml pull && npm run supa:down && npm run supa:up",

  "_comment_db": "—— BANCO DE DADOS ——",

```



```

    "db:backup":      "bash scripts/backup.sh",
    "db:restore":     "bash scripts/restore.sh",
    "db:backup:prod": "bash scripts/backup.sh prod",
    "db:restore:prod": "bash scripts/restore.sh prod",
    "db:shell":       "docker exec -it r2-postgres psql -U r2-user -d r2-
database",
    "db:shell:schema": "docker exec -it r2-postgres psql -U r2-user -d r2-database
-c 'SET search_path TO \"r2-schema\";'",
    "db:volume:inspect": "docker volume inspect r2-pgdata",
    "db:volume:list":    "docker volume ls | grep r2",

    "_comment_health": "—— SAÚDE E DIAGNÓSTICO —————",
    "health":          "bash scripts/healthcheck.sh",
    "health:db":        "docker exec r2-postgres pg_isready -U r2-user -d r2-
database",
    "health:containers": "docker ps --filter name=r2 --format 'table
{{.Names}}\\t{{.Status}}\\t{{.Ports}}'",
    "health:stats":     "docker stats --no-stream --filter name=r2",
    "health:disk":      "docker system df",

    "_comment_misc": "—— UTILITÁRIOS —————",
    "clean:images":     "docker image prune -f",
    "clean:all":        "docker system prune -f",
    "env:check":        "dotenv-cli -e .env.dev -- printenv | sort"
  },
  "devDependencies": {
    "cross-env": "^7.0.3",
    "dotenv-cli": "^7.0.0"
  }
}

```

6. Scripts Shell

6.1 scripts/backup.sh

```

#!/bin/bash
# Uso: bash scripts/backup.sh [prod]
# Sem argumento: usa .env.dev | com 'prod': usa .env.prod.local

ENV=${1:-dev}
ENV_FILE=".env.$ENV"
[ "$ENV" = "prod" ] && ENV_FILE=".env.prod.local"

set -o allexport
source $ENV_FILE

```

```
set +o allexport

CONTAINER="r2-postgres"
BACKUP_DIR="./backups"
TIMESTAMP=$(date +%Y%m%d_%H%M%S)
FILE="$BACKUP_DIR/r2-database_${ENV}_${TIMESTAMP}.sql"

mkdir -p $BACKUP_DIR

echo "=> Backup iniciado: $FILE"
docker exec $CONTAINER pg_dump \
  -U $DB_USER \
  -d $DB_NAME \
  --schema=$DB_SCHEMA \
  --no-owner \
  --no-acl \
  > $FILE

if [ $? -eq 0 ]; then
  gzip $FILE
  echo "=> OK: ${FILE}.gz ($(du -sh ${FILE}.gz | cut -f1))"
else
  echo "=> ERRO: backup falhou"
  exit 1
fi
```

6.2 scripts/restore.sh

```
#!/bin/bash
# Uso: bash scripts/restore.sh [arquivo.sql.gz] [prod]

FILE=${1:?Informe o arquivo de backup: bash scripts/restore.sh backup.sql.gz}
ENV=${2:-dev}
ENV_FILE=".env.$ENV"
[ "$ENV" = "prod" ] && ENV_FILE=".env.prod.local"

set -o allexport
source $ENV_FILE
set +o allexport

CONTAINER="r2-postgres"

echo "=> Restore de $FILE no banco $DB_NAME (schema: $DB_SCHEMA)"
read -p "    Confirmar? [s/N] " confirm
```

```
[ "$confirm" != "s" ] && echo "Cancelado." && exit 0

if [[ $FILE == *.gz ]]; then
    gunzip -c $FILE | docker exec -i $CONTAINER psql -U $DB_USER -d $DB_NAME
else
    cat $FILE | docker exec -i $CONTAINER psql -U $DB_USER -d $DB_NAME
fi

[ $? -eq 0 ] && echo "=> OK: restore concluído" || echo "=> ERRO: restore falhou"
```

6.3 scripts/healthcheck.sh

```
#!/bin/bash
# Verifica saúde de todos os containers r2 e do banco

RED='\033[0;31m'
GREEN='\033[0;32m'
YELLOW='\033[1;33m'
NC='\033[0m'

echo ""
echo "=====
echo "  R2 - Health Check"
echo "=====

# — Containers —————
echo ""
echo "Containers:"
docker ps --filter name=r2 --format "  {{.Names}}: {{.Status}}"

# — PostgreSQL —————
echo ""
echo "PostgreSQL:"
docker exec r2-postgres pg_isready -U r2-user -d r2-database > /dev/null 2>&1
if [ $? -eq 0 ]; then
    echo -e "  Status: ${GREEN}OK${NC}"
    SIZE=$(docker exec r2-postgres psql -U r2-user -d r2-database -t -c \
        "SELECT pg_size_pretty(pg_database_size('r2-database'));" | xargs)
    echo "  Tamanho DB: $SIZE"
    TABLES=$(docker exec r2-postgres psql -U r2-user -d r2-database -t -c \
        "SELECT count(*) FROM information_schema.tables WHERE table_schema='r2-
schema';" | xargs)
    echo "  Tabelas em r2-schema: $TABLES"
    CONNS=$(docker exec r2-postgres psql -U r2-user -d r2-database -t -c \
        "SELECT count(*) FROM pg_stat_activity WHERE datname='r2-database';" | xargs)
```

```

    echo "  Conexoes ativas: $CONNS"
else
    echo -e "  Status: ${RED}FALHOU${NC}"
fi

# — Recursos —————
echo ""
echo "Recursos (containers r2):"
docker stats --no-stream --filter name=r2 \
    --format "  {{.Name}}: CPU {{.CPUPerc}} | MEM {{.MemUsage}}"

# — Volume —————
echo ""
echo "Volume r2-pgdata:"
docker volume inspect r2-pgdata --format "  Mountpoint: {{.Mountpoint}}"
2>/dev/null \
    || echo "  Volume nao encontrado"

echo ""
echo "=====
```

7. Comparativo de Ambientes

Aspecto	Dev (Windows)	Prod Local (Linux)	Prod Supabase
PostgreSQL	Container, 5432 exposto	Container, rede interna	Supabase cloud
Linguagens	Host nativo	Containers	Containers
DB_NAME	r2-database	r2-database	postgres (Supabase)
Schema	r2-schema	r2-schema	r2-schema
DB_USER	r2-user	r2-user	postgres (Supabase)
Host DB	localhost:5432	postgres:5432 (DNS)	db.<ref>.supabase.co
Porta exposta	5432 → host	Nenhuma	Nenhuma (TLS Supabase)
Env file	.env.dev	.env.prod.local	.env.prod.supabase
Script start	npm run dev:up	npm run prod:up	npm run supa:up

8. Segurança e Boas Práticas

- Nunca commitar arquivos .env — todos no .gitignore.
- Senhas geradas com: `openssl rand -base64 32`
- Em prod, preferir variáveis de ambiente do CI/CD (GitHub Actions secrets, etc.) injetadas em runtime.
- SUPABASE_SERVICE_KEY exclusivamente server-side. Jamais exposta no bundle do cliente.
- PostgreSQL local em prod sem porta exposta — acesso externo somente via nginx na 80/443.
- Healthcheck no `depends_on` garante ordem de inicialização: banco pronto antes das apps.
- Volumes nomeados (r2-pgdata) isolam dados do filesystem do host e sobrevivem a docker down.
- Para Supabase, habilitar RLS (Row Level Security) em todas as tabelas públicas.
- Rotacionar senhas do banco periodicamente — atualizar .env e reiniciar containers.

9. .gitignore

```
# Secrets — nunca versionar
.env.dev
.env.prod.local
.env.prod.supabase
.env*.local
*.env

# Backups
backups/

# Node
node_modules/
```

10. To Do — Checklist de Implantação

10.1 Setup Inicial

Executar uma vez na criação do projeto:

- [] Criar estrutura de diretórios conforme seção 2 — `mkdir -p apps/nextjs apps/python apps/rust scripts backups`
- [] Criar os arquivos .env.dev, .env.prod.local e .env.prod.supabase a partir dos modelos da seção 3

- [] Adicionar todos os .env ao .gitignore antes do primeiro commit
- [] Criar scripts/init.sql com CREATE SCHEMA e GRANT (seção 4.1)
- [] Instalar dependências: `npm install -D cross-env dotenv-cli`
- [] Adicionar os scripts ao package.json conforme seção 5
- [] Tornar scripts executáveis: `chmod +x scripts/*.sh`
- [] Testar subida do ambiente de desenvolvimento: `npm run dev:up`
- [] Verificar criação do schema: `npm run db:shell:schema`
- [] Executar primeiro backup de teste: `npm run db:backup`

10.2 Produção Servidor Local

- [] Instalar Docker Engine no Linux (não Docker Desktop)
- [] Criar .env.prod.local no servidor (nunca via git)
- [] Configurar nginx.conf com virtual hosts e proxy_pass
- [] Configurar certificado SSL: `certbot --nginx` ou similar
- [] Subir produção: `npm run prod:up`
- [] Validar saúde: `npm run health`
- [] Configurar cron job de backup automático (diário):

```
# crontab -e
0 2 * * * cd /caminho/projeto && npm run db:backup:prod >> /var/log/r2-backup.log
2>&1
```

- [] Testar restore a partir de um backup gerado
- [] Configurar monitoramento de disco (volume r2-pgdata)

10.3 Produção Supabase

- [] Criar projeto no Supabase (supabase.com/dashboard)
- [] Criar schema r2-schema no Supabase via SQL Editor:

```
CREATE SCHEMA IF NOT EXISTS "r2-schema";
-- Habilitar RLS nas tabelas após criação
```

- [] Copiar Project URL, anon key e service key para .env.prod.supabase
- [] Habilitar RLS em todas as tabelas públicas no Supabase Dashboard
- [] Configurar políticas RLS por tabela conforme regras de negócio
- [] Testar conexão: `npm run supabase:up` e `npm run health`
- [] Validar que SUPABASE_SERVICE_KEY não aparece no bundle do cliente
- [] Configurar backups automáticos no plano Supabase (Pro+) ou via pg_dump externo

10.4 Manutenção Contínua

- [] Atualizar imagem postgres periodicamente: `npm run prod:deploy`
- [] Rotacionar senhas a cada 90 dias — atualizar `.env` e reiniciar
- [] Monitorar tamanho do volume: `npm run health:disk`
- [] Revisar logs regularmente: `npm run prod:logs`
- [] Testar restore de backup mensalmente em ambiente de desenvolvimento

11. Referência Rápida de Comandos

Ação	Comando
Subir dev	<code>npm run dev:up</code>
Parar dev	<code>npm run dev:down</code>
Subir prod local	<code>npm run prod:up</code>
Deploy prod local	<code>npm run prod:deploy</code>
Subir prod Supabase	<code>npm run supa:up</code>
Backup dev	<code>npm run db:backup</code>
Backup prod	<code>npm run db:backup:prod</code>
Restore	<code>npm run db:restore -- backup.sql.gz</code>
Shell psql	<code>npm run db:shell</code>
Health geral	<code>npm run health</code>
Status containers	<code>npm run health:containers</code>
Stats CPU/MEM	<code>npm run health:stats</code>
Logs follow	<code>npm run prod:logs</code>
Reset dev completo	<code>npm run dev:reset</code>
Limpar imagens	<code>npm run clean:images</code>